Memory for VLA policies: trends and insights after three core papers and 97 surrounding works

> This is the synthesis part of awesome_memory_vla. The three core deep dives are EventVLA, TRACE and SAI; all 103 entries are listed in the README. Every number below is taken from the original paper or abstract. Numbers from different papers rest on different task suites and scoring rules and cannot be compared directly.

1. What is happening in the field

1.1 Timeline: the June–August 2026 burst

By submission month, the 97 non-background papers in this repository split as: 24 in all of 2025, 23 in January–May 2026, then **20 in June, 10 in July and 17 in August 2026**. All three core papers appeared in mid-June (TRACE June 12, SAI June 15, EventVLA June 18), in the same month as KEMO (June 22), μVLA (June 10), MemoryWAM (June 18), WeaveLA (June 16), PRISM/ReMemBench (June 15) and the long-context diffusion-policy study (June 15). This is not coincidence: RMBench (March 1), RoboMME (March 4) and RoboMemArena (May 11) landed in spring and gave method papers a shared target, and the summer papers are the first round of answers.

1.2 A shared starting point

Nearly every first paragraph says the same thing: VLAs are Markovian (`a_t = π(o_t, l)`) and manipulation is not. The papers differ in their diagnosis of *why not*, and the diagnosis determines the memory design:

Diagnosis	Typical phrasing	Representative work	Design tendency
Evidence vanishes	Lift a cover, glimpse, close it; the cue leaves the view	EventVLA, TRACE, KEMO, UniMem	Sparse, explicit, addressable
State aliasing	Visually identical states at different stages (2nd vs 3rd button press)	KEMO, TFP, ChainVLA, WeaveLA, CAMP	Task-progress belief, event counting
Spatial forgetting	A single wrist camera loses objects that leave the view	AtlasVLA, AnchorVLA4D, SAM2Act+	Persistent spatial state, initial anchor frame
Partner unobservable	The partner's phase can only be inferred from history	SAI, Duet	History token + training-distribution coverage
No progress tracking	A frozen VLA executes chunks open-loop without knowing where it is	AGM, HELM, BATON, HyMeS	External symbolic state + verifier

1.3 Five technical families

The 97 papers fall into five families by *where memory lives and in what form* (README sections 3–7):

1. **Event / keyframe memory** (EventVLA, KEMO, Keyframe-Chaining, UniMem, WeaveLA, Bi-HIL, MemoryWAM's anchor layer, MemER's high-level frame selection): write only when something happened, store raw frames or their tokens.
2. **Dense compressed history** (MemoryVLA / MemoryVLA++, ContextVLA, CronusVLA, HAMLET, NativeMEM, StreamPI, TempoFit, DySta, FibVLA, PRISM, HALO, LaMem-VLA, Remember Smarter): every frame enters; cost is controlled by compression, tokenisation, KV reuse or sampling.
3. **Latent state / recurrent / slot memory** (TRACE, μVLA, VPWEM, VQ-Memory, MemoAct, RB-VLA, MTIL, DSSP, Chronos, TFP, CAMP, GMP, ELMUR, AEM, FM-VLA): history compressed into fixed-size vectors or slots updated recursively.
4. **Dual-system / agentic / symbolic memory** (MEM, MemER, MAP-VLA, Notes-to-Self, Explicit Language Memory, CodeGraphVLP, HyMeS, HELM, Harness VLA, AGM, PonderPounce, BATON, OnEvoMemory, ChainVLA): a VLM / LLM / code layer maintains text, graphs or progress pointers; a low-level VLA executes.
5. **Memory inside world models** (MemoryWAM, MemoryVAM, TriVLA, the imagination branch of MemoryVLA++): memory serves prediction, prediction serves action.

A sixth direction, **partner memory in multi-robot systems** (SAI, HATS, Duet, Tri-Manual, AutoIntervene), has only a handful of papers but shares the same mathematics with the other five.

2. The design space: six questions every memory VLA must answer

Question	EventVLA	TRACE	SAI	Other typical answers
What is stored (content)	Raw image frames	512-D latent of vision + state	GRU hidden state	One token per frame (NativeMEM), K/V (TempoFit), text (MEM, Notes-to-Self), semantic graph (CodeGraphVLP), force series (FM-VLA), voxel state (AtlasVLA)
When to write (trigger)	Learned future-keyframe probability + NMS + cooldown	Every step; gates decide how much	Every step	Kinematic + visual rules (KEMO), event classifier (UniMem), subgoal completion (WeaveLA), advance only after physical verification (AGM), value-guided (OnEvoMemory)
Where to write / read (addressing)	Temporal concatenation, attention finds it	Path signature of the robot trajectory	None (single vector)	Content similarity (MemoryVLA, HALO), progress-aware query (Keyframe-Chaining), trajectory-similarity retrieval of demos (MAP-VLA), LRU (ELMUR)
How much (capacity)	5 frames + 3–4 anchors	K = 4/6 slots	30 steps	Fixed token count (VPWEM, μVLA's m tokens), linear growth (frame stacking, StreamPI), unbounded text (MEM)

Question	EventVLA	TRACE	SAI	Other typical answers
Where it plugs in (integration point)	Input to the VLM vision encoder	Adapter before the action head	Decoder input	Action-expert conditioning (TFP, WeaveLA, FM-VLA), KV cache (TempoFit), prompt (MAP-VLA, PonderPounce)
What supervises it	Qwen3-VL keyframe labels + soft targets + curriculum	No labels; three stabilisers	Data curriculum + interventions	Time-contrastive (HAMLET), past-token prediction (PTP), VQA distillation (HALO), TBPTT (μVLA), imitation loss only (most)

How to read the table: the two extremes were claimed in the same month by two papers. EventVLA puts its weight on *when to write*; TRACE on *where to read*. The middle ground, learning both the write trigger and the addressing, is still empty.

3. Evidence across benchmarks

Publicly reported numbers on the three 2026 benchmarks (each paper's own protocol):

RMBench (9 tasks, RoboTwin 2.0)

Method	Avg.	Note
$\pi_{0.5}$ (no memory)	10.4–11.2	As reported by EventVLA and Chronos
MemoryVLA (QwenOFT)	41.7	EventVLA's reproduction
Mem-0 (RMBench's modular policy)	42.0	
ChainVLA	62.8	1.2B; 11.2 without Motion Tail, 3.0 without Progress Context
EventVLA (anchors only)	67.8	KEM not enabled
Chronos	73.6	10× fewer parameters than $\pi_{0.5}$, 30× fewer than Mem-0

RoboMME (16 tasks; temporal / spatial / object / procedural memory; $\pi_{0.5}$ backbone)

Method	Number
$\pi_{0.5}$ current-observation	17.93
FrameSamp+Modul (RoboMME's strongest variant)	44.51 (57.88 with 9× data)
PonderPounce 0.8B / 9B	50.04 / 60.83 (75.54 with 9× data)
WeaveLA	Hardest repetition slice SwingXtimes N=3: 0% → 47.8%; single-execution episodes unchanged
AGM	Beats the strongest memory baseline on the Counting subset (PickXTimes, BinFill)

RoboMemArena (26 tasks, > 1000 steps on average, 68.9% of subtasks memory-dependent)

Method	Cumulative / task success
$\pi_{0.5}$	52.5 / 41.3

Method	Cumulative / task success
PrediMem (the benchmark's dual-system policy)	4.5 / 14.5 below HyMeS
HyMeS ("skills in weights, memory in code")	66.2 / 60.1
BATON	+14.9 / +11.6 over SoTA

MIKASA-Robo (32 tasks, RL lineage)

Method	Number
MemoryVLA	41.2 (+11.8 over CogACT / π_0)
VPWEM	> 20% over diffusion policies and VLAs
μVLA (minimal recurrence)	Five training tasks 0.42 → 0.84; held-out 0.07 → 0.23
ELMUR	Best on 21 of 23 tasks; aggregate about 70% above previous best

Real-robot delayed / transient evidence

Paper	Number
EventVLA (ARX ACONE, 4 tasks × 20)	90 / 60 / 90 / 75 vs π_MEM 50 / – / 30 / 40
TRACE (5 tasks × 25)	ACT 25.50 → 69.23, DP 25.00 → 59.53, $\pi_{0.5}$ 50.47
KEMO (dual-arm, 830–2846 steps)	Task success +23.6, stage completion +34.1 over $\pi_{0.5}$
NativeMEM	Simulation 32.4 → 84.0, real robot up to 98.7, competitive with 20% of the data
UniMem	Simulation 93.4 vs 68.2 for fixed-interval sampling; hardware 80.0 vs 43.5 for hierarchical baseline
Chronos (single RGB, dual-arm)	78% over 4 tasks, 72% on the memory-dependent subset; $\pi_{0.5}$ 7% / 0%

Three observations. First, on RMBench "initial frame plus recent window" already reaches 67.8%, so that benchmark's memory demand is shallow, and Chronos's 73.6% comes from a full-history SSM rather than explicit event memory. Second, RoboMME and RoboMemArena are led by dual-system / agentic methods (PonderPounce, HyMeS, BATON): when the task needs **counting and procedural memory**, symbolic progress state is currently more reliable than end-to-end latents. Third, on real transient-evidence tasks the highest absolute success comes from end-to-end event memory (EventVLA, KEMO, UniMem). **No single memory leads on every benchmark**; RoboMME's conclusion that representation effectiveness is highly task-dependent has been confirmed by a full year of results.

4. Ten insights

Insight 1: the frontier moved from “how much to store” to “when to write”

The 2025 memory VLAs competed on window length and compression (CronusVLA, ContextVLA, MemoryVLA). The mid-2026 papers turn almost simultaneously to **write timing**: EventVLA learns future-keyframe probabilities, KEMO detects events from kinematics plus visual change, UniMem trains an event classifier, WeaveLA fires on subgoal completion, MemoryWAM uses event-boundary anchor frames, TFP’s mechanistic analysis finds write gain about 6× larger near manipulation events than in non-event phases, OnEvoMemory learns what to keep from rollout outcomes. Different architectures, one conclusion: **most of history is redundant with the present; the frames worth writing are the moments of state transition.**

Insight 2: “Present but Not Remembered” supplies the mechanism

Liao & Cao audit three frozen VLAs with layer-resolved linear probes and causal interchange interventions: past-frame content is linearly decodable throughout the network, but **information unique to the history, absent from the current frame, is nearly missing**; stored history is largely a copy of the present, and it is used causally only when the current frame is heavily degraded. This explains why frame stacking gives inconsistent gains (ContextVLA’s starting point) and why sparse event frames work: an event frame carries exactly what the current frame lacks. It yields a design rule for memory augmentation: **inject information unique to the past, not more history.** TRACE’s signature increment `δ_t`, CAMP’s compressed action history and PTP’s past-token prediction all read as instances of this rule.

Insight 3: raw frames or latents depends on how many bits must be recovered

EventVLA’s implicit-bank ablation (24.9% vs 75.2%) and TRACE’s success with 512-D slots (69.23) look contradictory; they measure two kinds of information. EventVLA’s tasks require remembering the colours, positions and order of 3–5 objects at once: high entropy, spatial detail. TRACE’s tasks require “which side” or “which book”: one or two bits of branch variable. RoboMME’s finding (representation effect depends on the task) becomes operational: **estimate how much information the downstream decision must recover from the past, then choose the representation.** Detail-heavy recall wants raw frames or dense tokens (EventVLA, NativeMEM, UniMem’s dense spatial memory); branch variables want slots or beliefs (TRACE, TFP, RB-VLA); counting wants symbols (AGM’s progress pointer, Notes-to-Self’s scratchpad).

Insight 4: addressing is the under-rated axis

Most work defaults to “concatenate in time and let attention find it”. TRACE is one of few papers that treats addressing as a first-class problem: trajectory signatures as keys, route similarity around 90 under order-preserving transforms and 37.8 under reversal. Keyframe-Chaining’s progress-aware query, MAP-VLA’s trajectory-similarity retrieval and ELMUR’s LRU slots are three other addressing schemes. The value of addressing is **a stable correspondence between write time and read time**: TRACE uses the path travelled, progress-aware methods use the stage reached. When a task’s branch structure does not

correspond to robot motion (the cue is only a colour, motions are identical), trajectory addressing degrades; that boundary is the next thing to solve.

Insight 5: long context is less brittle than claimed, but spurious correlation is real

Agarwal et al. (long-context diffusion policies) sweep context length systematically and conclude that naive scaling is not as brittle as the literature says: with UNet + cross-attention and ordinary data volumes, single-task policies reach high success on many tasks. The opposing evidence is equally solid: PTP shows diffusion policies *ignore* past–future action dependencies (the inverse of copycat), GMP finds that simply extending history drops performance through distribution shift and overfitting (gated memory +30.1 over long-history baselines on MemMimic), HALO needs VLM-prior distillation and sparse attention to suppress spurious correlations. The reconciliation: long context works when supervision steers attention toward task-relevant history. EventVLA's soft labels and curriculum, TRACE's balance / entropy / consistency losses and μVLA's TBPTT are all doing this job.

Insight 6: reliability comes from disciplined state updates, not capacity

AGM's thesis: if external memory treats "attempted" as "completed", a local execution error becomes a persistent task-state error, so the progress pointer advances only after physical evidence verifies the subgoal. The same discipline appears as EventVLA's NMS + cooldown (one event cannot flood the buffer), TRACE's gated writes (unrouted slots barely move), BATON's verifier agent (the VLA is invoked only after the wrist view confirms readiness) and HELM's state verifier (predict failure before execution). **A memory write should be a verified transaction**: a principle discovered independently on both the end-to-end and the agentic path.

Insight 7: dual systems still lead on counting and procedural memory

The leaders on RoboMME and RoboMemArena (PonderPounce, HyMeS, BATON) all have a high level that manages state and a low-level VLA that executes. HyMeS states it most bluntly: skills in weights, memory in code; motor skills come from imitation, memory-management heuristics from a coding agent iterating on rollout feedback. EventVLA's critique of dual systems (latency, error propagation) holds on transient-visual-evidence tasks, but for discrete procedural memory ("press three times", "skip the finished subtask") symbolic state is currently steadier. PonderPounce uses an MLLM's native causal context as memory and asynchronously passes only the newest cognition token (p50 78 ms refresh, 25 ms action), narrowing the latency gap. **The end-to-end versus dual-system line is shifting from "which is better" to "which information belongs at which level".**

Insight 8: the cost of memory is heading toward zero

EventVLA's multi-frame input cuts throughput from 2.91 Hz to 0.94 Hz, the most expensive in this batch; contemporaneous work heads the other way. TRACE adds 2.1 ms per step; NativeMEM reuses the VLA's own vision encoder to compress each frame per view into one token; TempoFit trains nothing and adds no tokens, reusing prefix K/V at intermediate layers; StreamPI adds zero parameters, using instruction-

anchored causal attention and length extrapolation for streaming multi-frame inference; DySta keeps one copy of static tokens and reuses their KV cache, 2× faster with higher success. **“Nearly free memory” is now an attainable engineering target**, and EventVLA’s threefold latency will be the first thing its successors optimise.

Insight 9: multi-robot collaboration is the same problem wearing another face

SAI treats the partly visible partner as a data-curriculum problem, yet its own ablation shows the history token is decisive for execution reliability (premature release 64.5% → 16.1%). Treat partner phase as a latent inferred from history and SAI’s setting is isomorphic to TRACE’s delayed evidence, with “early cue” replaced by “what the partner just did”. Current multi-robot work is almost entirely about data collection (HATS: one human plus an agent for four arms; Duet: human–human demonstrations for pretraining; Tri-Manual: offline re-timing), and **nobody has yet used an explicit memory module for partner-state estimation**. That is a clear gap.

Insight 10: evaluation is fragmenting while diagnostics mature

Within a year at least twelve memory-related benchmarks appeared: RMBench, RoboMME, RoboMemArena, RoboTwin-MeM, ReMemBench, LIBERO-Mem, MemMimic, MemoryRTBench, RuleSafe, Memory-T-Bench, MemoryBench, LongBench, with different task sets and scoring, so cross-paper numbers cannot be compared (Section 3 could only tabulate per benchmark). Meanwhile **diagnostics** are maturing: TRACE’s route similarity / branch consistency with an order-reversal negative control, Present-but-Not-Remembered’s probes and causal interventions, TFP’s hidden-state interventions, LongBench’s split of long-horizon failure into execution robustness and context dependence. The next round of evaluation should report both “success” and “what the memory actually used”.

5. Open problems

1. **Buffer saturation.** EventVLA’s five-frame FIFO evicts early evidence on > 10-minute event-dense tasks; TRACE’s fixed slots drop to 0.928 consistency at 2× history. Hierarchies (recent / event / gist, as in MemoryWAM) are the current compromise and have not been compared head-to-head with flat memories on one benchmark.
2. **Distinguishable addresses.** Trajectory signatures need branches to correspond to different motions; their dimension grows cubically with state dimension (17-D → 5219; depth 4 → 88,740). High-DoF humanoids and hands need log-signatures or learned low-dimensional trajectory keys.
3. **Supervision for the write trigger.** EventVLA labels offline with a 235B model, KEMO uses rules, UniMem a classifier, OnEvoMemory rollout outcomes. Which generalises across tasks has not been tested in a controlled way.
4. **Coupling between memory and chunk length.** EventVLA’s foresight collapses when the chunk shrinks from 50 to 15 (75.2 → 13.6); WhyChunking identifies non-Markovian expressivity as one of chunking’s benefits. Memory mechanisms and chunk length should be designed jointly.

5. **Safety of wrong memories.** AGM shows unverified memory freezes local errors into state; EventVLA does not discuss recovery from a wrongly committed frame.
6. **Explicit memory for multi-robot systems.** See Insight 9.
7. **Benchmark unification.** RoboMME's four memory types (temporal / spatial / object / procedural), RoboTwin-MeM's n-parameterisation and TRACE's stage-progress metric could be assembled into one reporting template.

6. What the three papers teach about method

- **Isolate a default and test it alone.** EventVLA changes only the memory representation (raw vs latent); TRACE changes only the addressing (with / without signature routing, with / without increments); SAI changes only the data curriculum (architecture fixed). The persuasiveness of all three comes from single-variable controls.
- **Report the cost.** EventVLA reports latency (0.94 Hz), TRACE reports milliseconds per step and parameter increments, SAI reports intervention data as a share of the baseline (30%). Readers can then judge whether the gain is worth it.
- **Diagnose instead of guessing.** TRACE's order-reversal negative control is the single most reusable design here: it turns "the memory uses history" from an assumption into a measurement.

7. Where we would go next (research directions)

1. **Foresight writes × trajectory addressing.** Use EventVLA's KEM head as the write trigger and TRACE's signatures as the slot address, and test whether performance holds on both RoboTwin-MeM (high-entropy visual evidence) and TRACE's tasks (low-entropy branch variables). Hypothesis: sparse writes solve capacity, trajectory addresses solve read/write alignment.
2. **Partner-phase memory.** In SAI's two-robot setting, replace A's 30-step GRU with a TRACE-style signature-routed slot memory (key = A's own trajectory, content = observations of B) and test whether Yield/Wait stays stable as B's delay grows.
3. **A memory-diagnostics benchmark.** Attach order-preserving and order-breaking history transforms to each of RoboMME's four task types and make route similarity / branch consistency a standard report item for every memory VLA.
4. **Cross-task transfer of write triggers.** On one backbone, compare EventVLA (learned), KEMO (rule-based) and UniMem (classifier) write policies on unseen tasks, to answer whether write timing is task-specific or general.